

Advanced Case Study: Smart Banking Transaction System

Background:

A digital banking system must support account creation, deposit, withdrawal, money transfer, transaction tracking, and analytics.

Requirements:

1. Use OOP principles (Abstraction, Encapsulation, Polymorphism).
2. Store accounts using ArrayList.
3. Implement custom exceptions.
4. Students must complete missing methods to pass JUnit tests.

Classes:

1. BankAccount (Abstract)
2. SavingsAccount
3. Transaction
4. BankService
5. Custom Exceptions
6. BankServiceTest

Students must implement:

- deposit()
- withdraw()
- transfer()
- getTotalBankBalance()

----- BankAccount.java -----

```
public abstract class BankAccount {

    protected int accountNumber;
    protected String accountHolderName;
    protected double balance;

    public BankAccount(int accountNumber, String accountHolderName, double balance) {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = balance;
    }

    public int getAccountNumber() { return accountNumber; }
    public double getBalance() { return balance; }

    public abstract void deposit(double amount);
    public abstract void withdraw(double amount) throws InsufficientBalanceException;
}
```

----- SavingsAccount.java -----

```
public class SavingsAccount extends BankAccount {

    public SavingsAccount(int accountNumber, String name, double balance) {
        super(accountNumber, name, balance);
    }

    @Override
    public void deposit(double amount) {
        // TODO: Validate amount > 0
        // TODO: Update balance
        // TODO: Add transaction
    }

    @Override
    public void withdraw(double amount) throws InsufficientBalanceException {
        // TODO: Validate amount > 0
        // TODO: Check sufficient balance
        // TODO: Deduct amount
        // TODO: Add transaction
    }
}
```

----- BankService.java -----

```
public class BankService {

    private List<BankAccount> accounts = new ArrayList<>();

    public void addAccount(BankAccount account) {
        accounts.add(account);
    }

    public BankAccount findAccount(int accountNumber)
        throws AccountNotFoundException {

        return accounts.stream()
            .filter(acc -> acc.getAccountNumber() == accountNumber)
            .findFirst()
            .orElseThrow(() -> new AccountNotFoundException("Account not found"));
    }

    public void transfer(int fromAcc, int toAcc, double amount)
        throws AccountNotFoundException, InsufficientBalanceException {

        // TODO:
    }
}
```

```

        // 1. Find both accounts
        // 2. Withdraw from sender
        // 3. Deposit to receiver
        // 4. Record transfer transaction
    }

    public double getTotalBankBalance() {
        // TODO: Use Stream API to sum balances
        return 0;
    }
}

```

----- BankServiceTest.java -----

```

@Test
void testTransfer() throws Exception {
    BankService service = new BankService();
    SavingsAccount acc1 = new SavingsAccount(1, "A", 1000);
    SavingsAccount acc2 = new SavingsAccount(2, "B", 500);

    service.addAccount(acc1);
    service.addAccount(acc2);

    service.transfer(1, 2, 300);

    assertEquals(700, acc1.getBalance());
    assertEquals(800, acc2.getBalance());
}

```